

Weighted Redistribution Algorithm

Data that is passively georeferenced gets heavily generalised when plotted on maps. This generalisation results from the fact that they are usually geocoded from information like place names, which might be ambiguous as it is impossible to pinpoint the precise location of the data using these names alone. This leads to the data often getting generalised to the centroid of the boundaries that represent the place names. Since the centroids are not the actual locations of the data, false hotspots will be created in heatmaps and they wouldn't be able to depict the real correlations or patterns in the data. This generalisation is an even bigger problem when the data involved is in multiple scales or levels. Huck, Whyatt and Coulton (2015) proposes a weighted redistribution algorithm, which can be used to disaggregate the passively georeferenced data from the centroids to locations which are more sensible and which can represent a better relationship or pattern in the data. No algorithm can relocate the passively georeferenced data to their actual locations as it is impossible to know them, but what this algorithm does is move them from the centroids of the polygons to a better location in the polygon based on values from the weighting surface. This algorithm does so by iterating through each polygon in each level or scale of the data, and then getting all the data points that are within the polygon. For each of the points within the polygon, random points inside the polygon are generated of which the random point having highest weight becomes the new seed location of the data point. Instead of plotting the point in the seed location, the algorithm makes a likelihood region around the seed location where the point may lie which shows the spatial ambiguity due to the same reason as to why the exact location of the points are impossible to find. All the points relocated this way shows a more real relationship as the points tend to cluster in regions of high weight which is a better and more believable pattern.

For this assessment, a random sample of 1,023 level 3 tweets relating to the Royal Wedding that have been geocoded to districts in Greater Manchester has been used along with the population density of Greater Manchester as the weighting surface to make a python implementation of the algorithm proposed by Huck, Whyatt and Coulton (2015). The tweets and administrative areas data which are vector files are read and converted to the same coordinate reference system of British National Grid, as this is necessary for the required topological operations to be done on them. The raster data of the weighting surface is then read using the open function from rasterio library. Since the tweets data at only one level is considered here, this algorithm first iterates through all the administrative areas (admin) at the county level dataset using a for loop and gets all the tweets that are in each of them. The within method which creates a spatial index of the tweets data is used here to locate the tweets within each administrative area.

```
#get the points inside admin
points = pointData.loc[pointData.within(admin)]
```

Now the algorithm takes on each tweet from the tweets within the administrative area using a for loop and for every tweet, random points are generated using the uniform class from numpy library which generates uniform random points inside the bounding box of the administrative area.

```
#get random points in admin

x = uniform(low=minx, high=maxx)
y = uniform(low=miny, high=maxy)
```

A while loop is then used to make sure the random points generated are according to the influence of the weighting surface (w) set by the user and a conditional statement is used inside the while loop to make sure all the random points generated are inside the administrative area. For every random point thus generated, the weight at their location is found from the weighting surface and the point that has the highest weight becomes the new seed location for that particular tweet. Now a likelihood region is created around the seed location, by creating a radius using the level of spatial ambiguity (s) set by the user. The radius is calculated using the equation:

$$r = \sqrt{\frac{As}{\pi}}$$

where A is the area of the administrative area (Huck, Whyatt and Coulton, 2015). This radius is then converted into pixels, which is then used with the disk function from the skimage library to create a disk with the seed location as the centre. The disk function is used here instead of using the circle_perimeter and line function because there can be a lot of overlapping pixels near the centre when you make lines from the centre to the perimeter. A column_stack function along with for loop is used then used to get the row and column values of all the pixels inside the disk.

```
#create a disk and get the row and column values of all the pixels in
it
for r, c in column_stack(disk((row,column), radius_px)):
```

This disk forms the likelihood region around the seed location. The values in the pixels inside this disk are set according to the equation:

$$v = 1 - \frac{\sqrt{(seed_x - cell_x)^2 + (seed_y - cell_y)^2}}{r}$$

where $seed_x$, $seed_y$, $cell_x$, $cell_y$ are the x and y locations of the seed and cell pixels (Huck, Whyatt and Coulton, 2015). So, the row and column values obtained from `column_stack` and `disk` functions are then converted to coordinate space and used in the equation.

The w and s values are set at 20 and 0.01 in this work as suggested by Huck, Whyatt and Coulton (2015). Increasing the value of w increases the number of random points generated, which will result in clusters forming only in high population density areas while lowering it can create more false hotspots. Changing the s values changes the radius of the likelihood region and 0.01 has been found to apt for the level we are working on. Most of the algorithm has been implemented in the form of functions so that it is robust and exception handling has also been taken care of.

The plot of the tweets data set before and after applying the algorithm is shown below:

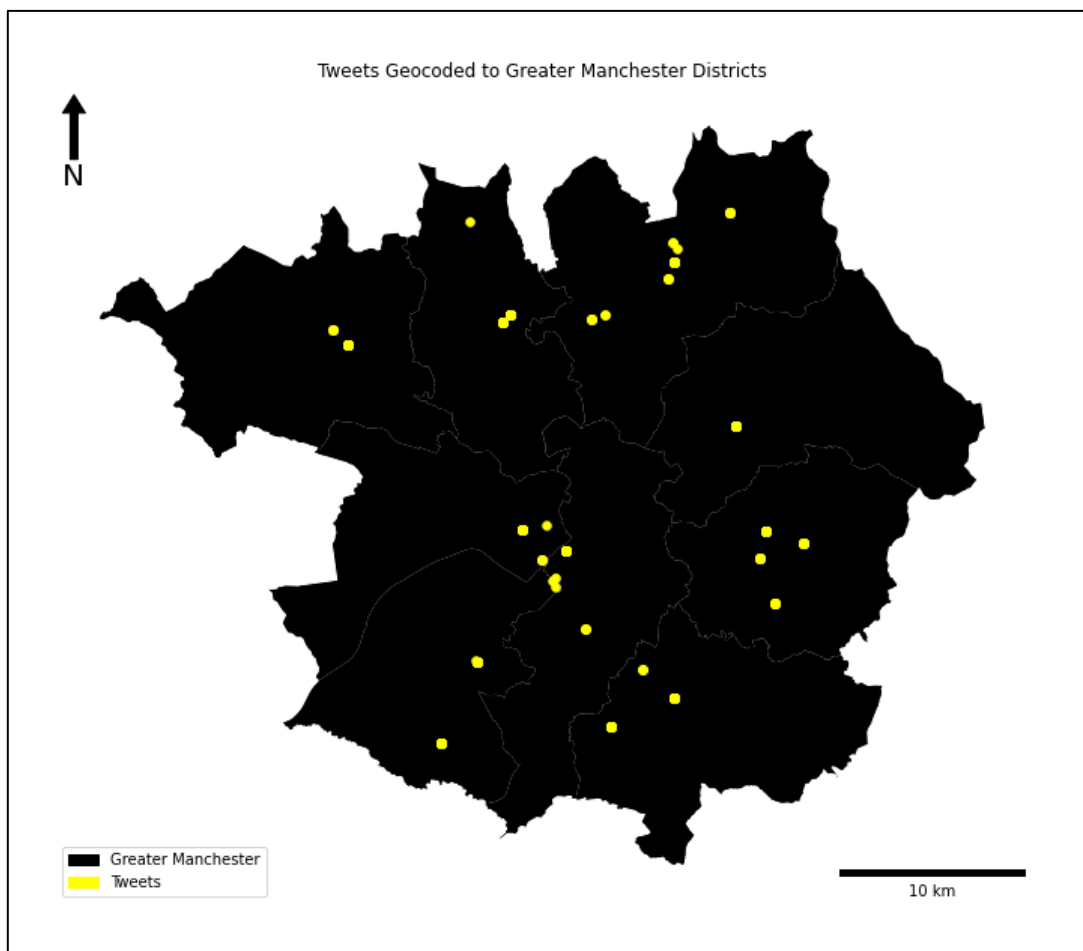


Fig 1: Tweets data plotted before applying the algorithm

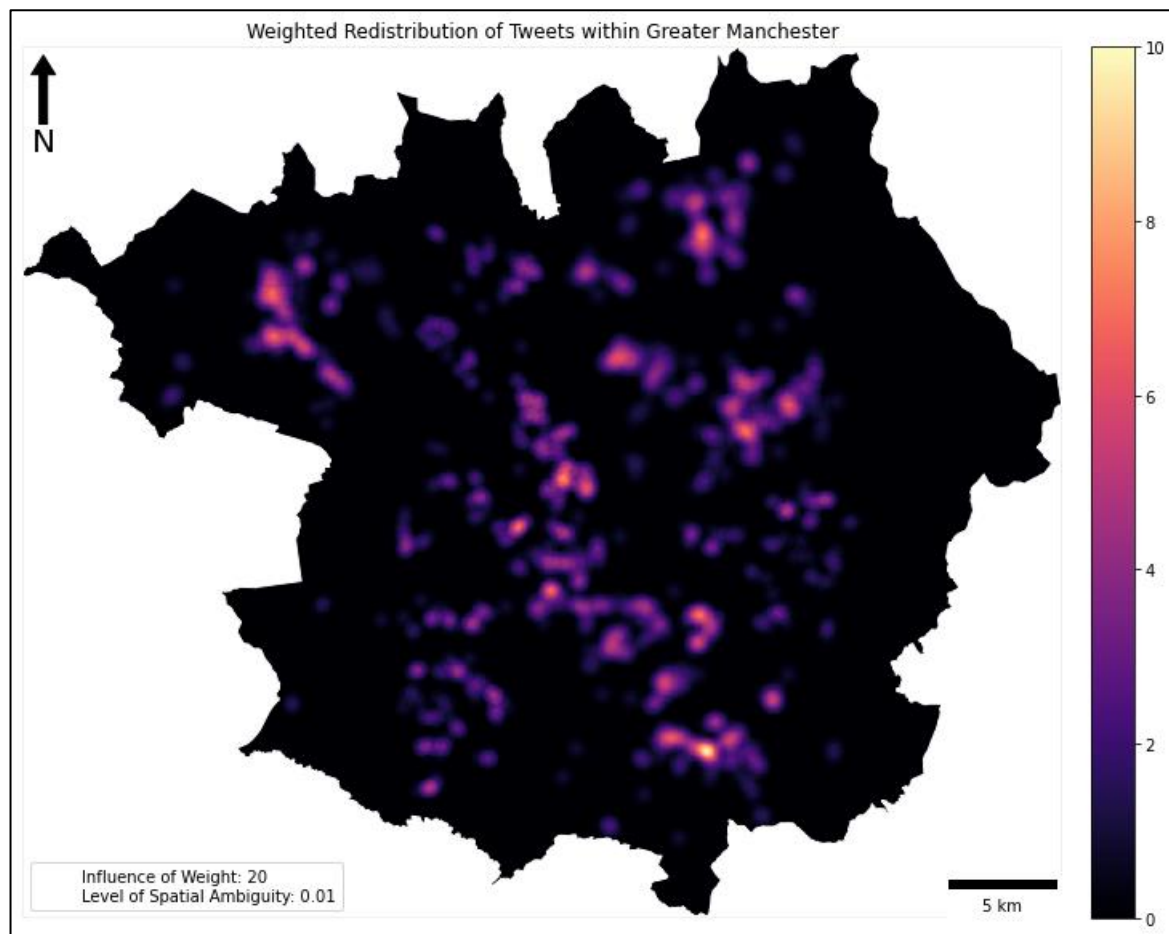


Fig 2: Tweets data plotted after applying the algorithm.

A limitation of this algorithm is that since we are working at level 3, it is impossible to go down to a lower hierarchy like towns or cities and find the relationships between the data points there. And every time the algorithm runs, different random points are generated resulting in a different distribution of the dataset and hence inconsistent maps are created. The distributed points are also not in any way at the right location, their locations just represent a better spatial arrangement than being generalised to centroids.

References:

Huck, J., Whyatt, D. and Coulton, P., 2015. Visualizing patterns in spatially ambiguous point data. *Journal of Spatial Information Science*, 2015(10), pp.47-66.